

Deployment Increment Review

QA and BA review of everything the team shipped between 12 and 17 September — two Dockerfiles and three new Terraform modules. What it means for testing, what it fixed, and the five new issues it surfaced.

AUTHOR	REVIEWED	RANGE	DAYS TO SUBMIT	
Shanthi Subramanian · BA / QA	17 September 2026	f6daee2...a816378 693bd95...9106553	13	

10	0	5	28	0
DEV COMMITTS	OF 6 DEFECTS FIXED	NEW ISSUES	NEW TEST CASES	APP CI PIPELINES

What changed

Ten commits across two branches. Entirely deployment and infrastructure — no application logic was touched.

Application branch — `release/1.00` (Suleman)

CHANGE	WHAT IT IS
Two Dockerfiles	Multi-stage builds for the Workbench (<code>1-API</code>) and Mock Systems (<code>6-MockExternalSystems</code>). SDK image publishes, <code>aspnet</code> runtime image runs, both listening on <code>:8080</code>
<code>.dockerignore</code>	Excludes <code>**/bin/</code> , <code>**/obj/</code> at any depth, plus <code>appsettings.Development.json</code> and <code>webapp/.env.local</code> so no local secret can be baked into an image
<code>ops/certs/</code>	A gitignored drop-point for a corporate proxy root CA, with a well-written README, so <code>dotnet restore</code> works behind SSL inspection
<code>ops/README.md</code>	Build and local smoke-test commands, and a documented end-to-end verification dated 2026-09-15

CREDIT WHERE IT IS DUE

Three things in this increment are genuinely good engineering. The `.dockerignore` excludes nested `obj/` at every depth with a written explanation of the Windows-host `project.assets.json` failure it prevents. The Workbench Dockerfile copies exactly the right project set — I verified every direct and transitive `ProjectReference` is present, and `webapp/` is excluded with a stated reason rather than by accident. And the developer recorded an actual end-to-end verification with a date, rather than asserting it works.

Infrastructure branch — `main` (Suraj)

MODULE	WIRED INTO DEV?
<code>terraform-azure-virtualnetwork</code> — VNet, subnets, NSGs, route tables, peering	PROVISIONED, UNUSED
<code>terraform-azure-containerregistry</code> — ACR, Standard SKU, <code>admin_enabled = false</code>	YES
<code>terraform-azure-containerappsenvironment</code> — CAE, system-assigned identity	YES

The dev environment now has `main.rg.tf`, `main.vnet.tf`, `main.acr.tf` and `main.cae.tf`. `.checkov.yml` gained six ACR-related suppressions, each with a written justification.

What did *not* change

- **No application code.** Every `src/` file except the two new Dockerfiles is byte-for-byte identical.
- `main` **still has no application.** The branch a judge lands on remains IaC-only; `release/1.00` has still never been merged.
- **Still no application CI.** Both pipelines remain path-filtered to `iac/environments/dev/*`. Nothing runs `dotnet test`, Playwright, or `docker build`.
- **IaC scans remain soft-fail.** Trivy and Checkov still warn rather than gate.

Open defects — status

Verified by diffing the defective code across the increment, not by re-running tests.

HEADLINE

None of the six open defects has been fixed. Each one's implementing code is unchanged across `f6daee2...` `a816378`. This is a factual statement about the diff, not an inference from test results.

ID	DEFECT	IMPLEMENTING FILE	STATUS
DEF-001	User directory readable with no credential	DevBypassAuthHandler.cs	UNCHANGED
DEF-002	No role enforcement; actor self-declared in body	Controllers/*	UNCHANGED
DEF-003	Unknown ids surface as HTTP 500	BaseApiController.cs	UNCHANGED
DEF-004	Empty assessment finalizes and reaches committee	AssessmentService.cs	UNCHANGED
DEF-005	<code>control</code> table never seeded	seed/	UNCHANGED
DEF-006	<code>func_getOrCreateAssessment</code> race	func_getOrCreateAssessment.sql	UNCHANGED

This is not a criticism of the increment — the work done was deployment work, and deployment is one of the six graded SDLC stages. But with **13 days to submission**, the two High-severity governance defects (DEF-002 and DEF-004) are the ones that contradict the product's own central claim, and they remain open.

Regression check

Because no application code changed, the existing 80-test suite remains valid without modification. No test needed updating for this increment, and no regression is possible from it. The suite was not re-run against `a816378` — with an empty `src/` diff there is nothing for it to detect, and claiming a fresh run would overstate the evidence.

New findings

Five issues raised by reviewing this increment. One is a deployment blocker.

DEF-007 HIGH DEPLOYMENT

There is no path to apply the database schema in Azure

The application is entirely stored-procedure driven — every repository method calls a named `func_*` routine. Those routines live in `src/4-Persistence/...AdminUI.DB/`, which is **a plain SQL folder, not a compiled project**. Three facts together make this a blocker:

- The Dockerfile never copies `...AdminUI.DB/` into the image — verified, zero references.
- `Program.cs` has no migration, bootstrap or `deploy_all.sql` execution on startup.
- No pipeline applies it — there is no application pipeline at all.

`ops/README.md` says the Flexible Server uses the "same schema `deploy_all.sql` already applies locally", but names no mechanism for applying it to Azure. On first deploy the containers will start cleanly and then fail on every request, because not one function exists.

IMPACT

First deployment to Azure cannot work. Discovered on deploy day, this costs hours the team does not have

FIX OPTIONS

A schema-deploy step in the release pipeline (psql against the Flexible Server), an init container, or a startup bootstrap guarded to run once

OWNER

Suraj (pipeline) with Suleman (schema)

The VNet is provisioned but connected to nothing

`main.vnet.tf` creates a VNet with two purpose-built subnets:

- `snet-cae-dev-01` — a `/20` carrying a `Microsoft.App/environments` delegation, which is exactly and only what a Container Apps Environment needs
- `snet-pep-dev-01` — a `/24` with service endpoints for Key Vault, Storage, SQL and Web, i.e. sized for private endpoints

The Container Apps Environment module *accepts* `infrastructure_subnet_id`, `internal_load_balancer_enabled` and `vnets_to_link`. `main.cae.tf` passes **none of them**, and `module.vnet` is referenced by no other file and no output.

So the network designed for isolation is paid for and unused; the Container Apps Environment remains publicly reachable. There is a second-order effect worth noting: the `.checkov.yml` suppression of `CKV_AZURE_139` (ACR public network access) is justified on the grounds that "agents have no private network path to the registry" — but a private-endpoint subnet now exists, unused, which weakens that justification.

IMPACT

Cost without benefit; the network-isolation story cannot be demonstrated

FIX

Pass the delegated subnet id into the CAE module, or state explicitly that VNet integration is out of scope and defer the VNet

OWNER

Suraj

Both containers run as root

Neither Dockerfile contains a `USER` directive, and `mcr.microsoft.com/dotnet/aspnet:10.0` defaults to root. Every process in both images therefore runs privileged.

This is the single most commonly flagged container finding, and both Trivy and Checkov detect it (`DS002` / `CKV_DOCKER_3`). Since the team already runs both scanners against Terraform, extending them to images would catch this automatically.

FIX

One line per Dockerfile — `USER $APP_UID`, which the .NET 10 base images define for exactly this purpose

OWNER

Suleman

DEF-010 MEDIUM OPERATIONS

Observability is wired in code but disabled in infrastructure

`main.cae.tf` carries this line:

```
# log_analytics_workspace_id = module.log_analytics_workspace... #disabled for now to avo
```

Both services already register `Azure.Monitor.OpenTelemetry.AspNetCore` and activate it whenever `APPLICATIONINSIGHTS_CONNECTION_STRING` is set. So the application side is ready and the infrastructure side is commented out — meaning a deployed environment would emit no logs, traces or metrics at all.

The cost concern is legitimate and was raised on the team's own architecture call. But **Operations is one of the six graded SDLC stages**, and "we wired the code and disabled the resource" is a weaker story than a free-tier workspace with sampling. Worth a deliberate decision rather than a commented line.

OWNER

Suraj, with a team decision on cost

DEF-011 LOW CONTAINER HYGIENE

No HEALTHCHECK, and base images are not digest-pinned

Neither image declares a `HEALTHCHECK`, and both reference floating tags (`sdk:10.0`, `aspnet:10.0`) rather than digests. Container Apps supplies its own probes, so the health check matters less in the target environment than it does for local smoke testing; the floating tag means two builds a week apart can differ silently.

FIX

Add `HEALTHCHECK` hitting `/api/Ping`; pin base images by digest once the build is stable

OWNER

Suleman

Testing strategy update

The increment adds two layers the existing strategy did not cover.

The strategy so far had four layers — unit, API/authorization, UI journey, AI evaluation — plus one acknowledged gap (database invariants). This increment introduces **container** and **infrastructure** as testable surfaces, and makes **deployment** a thing that can be verified rather than only described.

LAYER	STATUS BEFORE	NOW	TOOLING
Unit — service invariants	62 TESTS	Unchanged	xUnit + Moq
API + authorization	12 TESTS	Unchanged	Playwright
UI journeys	6 TESTS	Unchanged	Playwright
AI evaluation	2 OF 12	Still blocked on a model	Custom C# harness
Database invariants	GAP	Still the top gap	Testcontainers (<i>not started</i>)
Container image	—	NEW — 12 CASES	docker build/run, Trivy image scan
Infrastructure as code	partly, in the IaC pipeline	NEW — 11 CASES	terraform validate, tflint, Checkov, Trivy
Deployment verification	—	NEW — 5 CASES	Smoke tests against a deployed environment

Principles carried forward

The new layers inherit the same rules that govern the existing suites, because they solve the same failure modes:

- **Assert the correct behaviour, mark known gaps** — CT-07 (non-root) and IAC-08 (VNet integration) assert the secure state and are expected to fail until DEF-008/009 are fixed, so the fix announces itself.
- **Never let a dead environment look like a pass** — every container and deployment case must first confirm the thing under test is actually reachable, exactly as `global-setup.js` does for the API suite.
- **Keep counter-evidence** — CT-06 (no secret in the image) passing is what makes the DEF-009 finding precise rather than "the containers are insecure".

PRIORITY HAS NOT CHANGED

These new layers are valuable, but they do *not* displace the top recommendation. The `audit_event` append-only trigger and the `CHECK` constraints remain unreachable by all 80 existing tests and all 28 new ones — container and IaC tests do not touch application data. Testcontainers is still the single highest-value thing left to build.

New test cases

28 cases covering the new surface. None executed yet — Docker Desktop was not running at review time, so these are specified and ready, not run.

EXECUTION STATUS, STATED PLAINLY

Every case below is **Not run**. I could not build either image because the Docker daemon was unavailable during this review. The expected results are derived from reading the Dockerfiles and Terraform, which is enough to specify a test but *not* enough to claim a result. Where I predict a failure (CT-07, CT-08, IAC-08, IAC-11) that is a prediction from static analysis, and is labelled as such.

Container image – 12 cases

ID	CASE	STEPS / EXPECTED	PREDICTED
CT-01	Workbench image builds from repo root	<code>docker build -f src/1-API/.../Dockerfile -t humaid-workbench .</code> → exit 0	PASS
CT-02	Mock Systems image builds	Same pattern → exit 0	PASS
CT-03	Build fails clearly from the wrong context	Build from the project folder → fails with a comprehensible error, not a silent partial image	UNKNOWN
CT-04	Workbench container answers on :8080	<code>docker run -p 5211:8080</code> → <code>GET /api/Ping</code> returns "pong"	PASS
CT-05	Mock Systems container answers	<code>GET /api/ping</code> → 200	PASS
CT-06	No secret is baked into the image	Inspect all layers for <code>appsettings.Development.json</code> and <code>.env.local</code> → absent	PASS
CT-07	Container does not run as root	<code>docker run --rm humaid-workbench id -u</code> → not 0	FAIL – DEF-009
CT-08	Image declares a HEALTHCHECK	<code>docker inspect</code> → <code>Healthcheck</code> is non-null	FAIL – DEF-011
CT-09	No HIGH/CRITICAL OS vulnerabilities	<code>trivy image --severity HIGH,CRITICAL</code> → none	UNKNOWN
CT-10	Workbench reaches Mock Systems over the container network	Both on <code>ops_default</code> ; <code>GET /api/DataIngestion/MockCustomers</code> → 6 records	PASS
CT-11	Config comes only from env vars	Run with no env → fails loudly with a clear message, never silently defaults to a wrong DB	UNKNOWN
CT-12	Image size is reasonable	Runtime image, not SDK → under ~350 MB	PASS

Infrastructure as code – 11 cases

ID	CASE	STEPS / EXPECTED	PREDICTED
IAC-01	Formatting is clean	<code>terraform fmt -check -recursive</code> → exit 0	PASS
IAC-02	Dev environment validates	<code>terraform init -backend=false && validate</code> → success	PASS
IAC-03	TFLint reports no errors	<code>tflint --recursive</code> → exit 0	UNKNOWN
IAC-04	Every Checkov suppression carries a justification	Each <code>skip-check</code> entry has an adjacent comment → currently true for all 7	PASS
IAC-05	ACR admin account is disabled	<code>admin_enabled = false</code> → identity-based auth only	PASS
IAC-06	CAE uses a managed identity, not a stored credential	<code>system_assigned = true</code> ; no client secret anywhere	PASS
IAC-07	Every resource carries the six mandatory tags	business_unit, customer, environment, product, owner, region present on all	PASS
IAC-08	CAE is VNet-integrated	<code>infrastructure_subnet_id</code> resolves to the delegated <code>snet-cae-dev-01</code>	FAIL – DEF-008
IAC-09	The CAE subnet is correctly delegated and sized	<code>Microsoft.App/environments</code> delegation, <code>/20</code> ≥ the required <code>/23</code>	PASS
IAC-10	Plan contains no unexpected destroy or replace	<code>terraform plan</code> → review the destroy count on every PR	UNKNOWN
IAC-11	Log Analytics is wired to the CAE	<code>log_analytics_workspace_id</code> is set	FAIL – DEF-010

Deployment verification – 5 cases

ID	CASE	STEPS / EXPECTED	PREDICTED
DEP-01	Schema and functions exist in the target database	After deploy, <code>SELECT count(*) FROM pg_proc WHERE proname LIKE 'func_%'</code> → 40+	FAIL – DEF-007
DEP-02	Seed data is present	6 users, 4 FFIEC categories, 1 workflow rule	FAIL – DEF-007
DEP-03	The deployed Workbench answers	<code>GET https://<fqdn>/api/Ping</code> → 200	UNKNOWN
DEP-04	Mock Systems is not publicly reachable	It is an internal dependency; the public internet must not reach it directly	UNKNOWN
DEP-05	A rollback path exists and is documented	Previous image tag redeployable without data loss	NOT DEFINED

BA impact

What this increment changes for requirements, and what it now lets us pin down.

Non-functional requirements just became concrete

The NFR gap analysis previously had to be written in the abstract. With real infrastructure defined, four of those NFRs now have a specific, decidable answer — which is exactly the kind of requirement expansion the brief grades.

NFR	NOW DECIDABLE AS	STATUS
Data residency	All resources in <code>eastus2</code> ; the AI provider is Azure AI Foundry in-tenant rather than a third-party API	ANSWERED
Network isolation	VNet with delegated subnets exists — but nothing uses it (DEF-008)	DESIGNED, NOT ENFORCED
Secret management	No secret in any image; ACR admin disabled; system-assigned managed identity. Key Vault still referenced only in docs	PARTLY ANSWERED
Observability	Code instrumented; Log Analytics commented out (DEF-010)	DEFERRED ON COST
Availability / DR	Single region, zone redundancy off, no geo-replication — all deliberate and documented in the Checkov justifications	STATED DECISION
Data retention	Still undefined — no backup or retention policy on the Flexible Server	OPEN

A new open question for the team

Q6 — WHO OWNS DATABASE SCHEMA DEPLOYMENT?

DEF-007 is not purely technical; it is an unassigned responsibility. The schema is a first-class deliverable of the application (every repo method depends on a named function), but applying it to a deployed environment is a pipeline concern. Neither owner has claimed it, which is precisely why it fell through. This needs a named owner before the first Azure deploy, not during it.

Existing open questions — unchanged

Q3 (does a scoring override need secondary sign-off?) and Q5 (who approves configuration changes?) remain genuinely open and unanswered in code. Q1, Q2 and Q4 remain resolved. Nothing in this increment touched them.

Traceability

No user story's coverage changed, because no application behaviour changed. The register still stands at **12 of 22 covered, 6 partial, 4 with no coverage**. Deployment work does not move those numbers — a point worth making explicitly, because shipping Dockerfiles can feel like progress against the product requirements when it is progress against a different stage entirely.

Recommendations

Thirteen days to submission. Ordered by what protects the most score.

#	ACTION	WHY NOW	OWNER
1	Fix DEF-007 — schema deployment	Without it the first Azure deploy cannot work, and deploy day is the worst time to find out	Suraj + Suleman
2	Merge <code>release/1.00</code> into <code>main</code>	Raised on 12 Sep, still not done. Judges land on <code>main</code> and find no product. Also blocks every CI recommendation below	Suraj
3	Fix DEF-002 and DEF-004	The two High governance defects. 15% of the score is human-in-the-loop, and DEF-004 lets an empty assessment reach a committee vote	Suleman
4	Application CI pipeline	Testing is 1 of 6 graded SDLC stages and currently nothing runs automatically. Now also needs <code>docker build</code>	Suraj
5	Testcontainers for DB invariants	US-9.2 — the immutable audit trail — is the flagship claim and is proven nowhere	QA
6	Configure the AI provider and run the evals	10% of the score rests on 2 of 12 cases actually executing	Suleman + QA
7	DEF-008/009/010 — VNet, non-root, observability	Cheap fixes; DEF-009 is one line per Dockerfile	Suraj + Suleman

THE HONEST READ ON THIS INCREMENT

The deployment work is good and it advances a graded stage that had nothing in it. But across five days the defect count moved from six to eleven, and none of the original six closed. Two of the top three recommendations from 12 September — merge to `main`, application CI — are untouched, and both are blocking other work rather than merely pending.

What QA will do next, unless redirected

1. Build both images and execute CT-01 to CT-12 as soon as Docker is available — the container layer is specified and ready.
2. Start the Testcontainers suite for the audit trigger and `CHECK` constraints.
3. Fold these 28 cases into the main test case register once executed, so there is one register rather than two.

Reviewed against `origin/release/1.00` at `a816378` and `origin/main` at `9106553`, fetched 17 September 2026. Defect status was established by diffing each implementing file across the increment. The 28 new test cases are specified but not executed — Docker was

unavailable during this review, and that is stated rather than worked around. Companion documents: *QA & BA Strategy*, *QA Codebase Walkthrough*, *QA Test Case Register*.